CONVERSATIONS WITH CLAUDE

VOLUME THREE

PLEASE UPGRADE YOUR PLAN

by Suraaj Chandra Jha

a free book about handing the machine tools

THIS BOOK IS FREE

Take it. Give it away.

Free, like the first two. Copy it, mail it, teach from it. Keep it whole and keep my name on it. CC BY-NC-ND 4.0. If you feel you must pay someone, buy a beginner a coffee and open Volume One on their laptop.

Full screen. Press next.

Full screen. Press next. This volume gets its hands dirty: there is a terminal, one real command to type, and keys to keep secret. Everything is explained from zero. When the book says try it, try it.

Not the manufacturer.

Claude is made by Anthropic. This book is not: not affiliated, not endorsed, not reviewed. Product names, plans, and limits move fast in this volume's territory. The shapes and principles are durable. For today's exact numbers, go to the live documentation.

PREVIOUSLY

The story so far.

Previously: Volume One taught you to converse. Volume Two showed you the machinery: tokens, the window, the honest limits. You now know more than most daily users. This volume is where you stop being a user.

Ten doors.

CHAPTER 1

01

THE SLIPPERY SLOPE

Warning: this volume voids your excuse for busywork.

You will not go back.

Power users tell the same origin story. Nobody sets out to run a fleet of agents, AI helpers working unattended. One afternoon a boring chore, renaming eighty files, chasing the same weekly report, gets handed to the machine, and two hours become ten minutes. The chore was small. The taste is not. Once you have watched work occur, done correctly, while you sipped your tea, doing it by hand again feels faintly ridiculous. That afternoon rewires people. This volume is that afternoon, stretched across ninety pages, on purpose.

YOU:

can u rename all these photos to the date they were taken

CLAUDE:

Done. 214 files, oldest is March 2019.

* The divider page did warn you.

The ladder has five rungs.

This volume climbs a ladder, and you should see the whole of it before the first rung. Chat: you talk, it answers. Workspace: it makes things beside you. Tools: it reaches into your files and your calendar. API: you speak to the model directly, no app in between. Agents: you delegate whole jobs and judge the results. Each rung gives Claude more reach into your world, and each asks more judgment of you. That trade is the price of every rung, and the book climbs slowly enough to pay it.

* Skipping rungs is how ladders earned their reputation.

Power is reach, not intelligence.

Nothing in this volume makes Claude smarter. The model running your agents is the one from page one of Volume One. Only reach grows: what it may read, edit, run, and send in your name. An advisor in a phone booth is a curiosity. The same advisor holding your keys is a policy question.

* Phone booth: a glass box that held one caller and zero keys. Ask an elder.

Now it can break things for you.

Volume One made you a promise: there is no wrong button, and the worst outcome is a dull answer. Still true, in the chat box. No prompt hurts the model, and a fresh conversation is always a clean slate. But the promise carried fine print, and this is where it comes due. Once Claude can touch things, files, calendars, code that runs, its mistakes stop evaporating when the chat ends and start landing in your world. You still cannot break it. It can now break things for you, which is a different sentence. The rule that governs this arrives in chapter four: give keys slowly.

YOU:

wait did u actually delete those

CLAUDE:

No. I asked first. That habit is the entire safety model.

* The trash folder forgives. Not everything has a trash folder.

The oath: curiosity plus care.

Before the tools come out, a small oath, hand anywhere you like. Try everything in a sandbox first: a copy, a spare folder, a place where mistakes cost nothing. Read before you run: when Claude hands you a command, ask what it does until you could explain it to someone else. Verify before you ship: the checking habit from Volume One, now with consequences. None of this is caution as a tax on fun. The hacker ethic has always been curiosity plus care, and the reason old hackers exist is that they took both.

YOU:

whats the worst that could happen

CLAUDE:

In this folder? Nothing. Hence the folder.

* Oaths taken in lowercase still count.

Start where you already live.

The climb starts somewhere comfortable: the same chat window you have lived in for two volumes. It has been holding out on you. Behind the conversation sits a workshop, and the machine you have only ever heard talk can hand you finished things: a document, a diagram, a small working app, built beside the chat while you watch. No terminal yet (the bare typing window programmers use), no keys to guard. The first rung has been under your feet the whole time. Turn the page and ask it to make something.

* Chapter two opens with a tip calculator. It does not stay one.

CHAPTER 2

02

CLAUDE AT WORK

It makes things now. Say thank you.

It can hand you a thing.

Until now, Claude has answered you in words. Ask it for a web page and it could only hand you the code, a recipe with no kitchen. Then Anthropic gave it a workbench. Ask now and a panel opens beside the chat holding the actual page: working buttons, real layout, live. These things are called artifacts: documents, diagrams, small apps, even games, built while you watch. The chat window became a workshop. You did not learn a new skill to earn this. You asked, the same way you always have.

YOU:

make me a tip calculator. nothing fancy

CLAUDE:

Done. It is running beside this chat. Type a bill.

* You can share the link. The recipient does not need to know how easy it was.

You direct. It rebuilds.

The thing in the panel is not a final answer. It is a first draft that happens to run. Ask for dark mode and it recolors itself. You give notes, the way a director gives notes, and the machine does the carpentry. Even ‘less like a tax form’ is a valid instruction.

* The director never holds the hammer. Some of them miss it.

Your first piece of infrastructure.

Volume Two introduced projects as a memory trick: a folder of chats that share context. Now use one as a workshop. Load a project with your style guide, your boilerplate (the text you reuse everywhere), your three best examples of the thing you make. Every conversation inside starts already briefed, like a new hire who actually read the onboarding docs. You stop re-explaining yourself, which you have been doing since page one of this book. Set it up once, benefit every day after. That is infrastructure, and this piece takes about ten minutes.

* It rereads the whole briefing every single morning and never once sighs.

The window starts filling itself.

So far you have carried everything to Claude by hand: paste the email, paste the calendar, paste the doc. Connectors reverse the flow. With your permission, it can read your calendar and query the tools your work lives in. The context window from Volume Two still rules everything. The difference is who does the fetching. Every connection is a key you chose to hand over, and convenience and exposure always arrive in the same envelope. Connect what you would show a trusted assistant, and stop there.

YOU:

check my calendar, im trying to book the dentist thursday

CLAUDE:

Clear after 2. Go get your teeth judged.

* The permission screen is not a formality. Read it like a lease.

Files in, files out.

Hand it a spreadsheet and ask what is strange about the numbers. Hand it a forty-page PDF and ask for the two paragraphs that matter. Then ask for the answer back as a slide deck, a formatted document, or a cleaner spreadsheet than the one you gave it. None of this is glamorous, which is why it is the most used power feature in the building. The office was always a machine for turning documents into other documents. It just found a much faster gear.

* It also reads page 38, where they hid the fee. It always reads page 38.

When chat is the wrong shape.

There is a moment worth catching. You are pasting the same kind of thing into the same kind of prompt for the third Monday in a row. Or you want one change made across a whole folder, and the chat box takes files one at a time. Or you catch yourself wishing the work would happen while you slept. These are signs you have outgrown the room. Chat is a shape: one person, one window, one exchange at a time. Some jobs need a different shape. The rest of this volume is those shapes.

YOU:

perfect. ok now the same thing for the other 60 files

CLAUDE:

Happily. Paste the next one.

* The box did not shrink. You grew.

Do not fear the black window.

Everything in this chapter happened inside a browser tab, in an app someone furnished for you. Comfortable and carpeted. But the itch you just named does not live there. To scratch it, Claude has to sit on your machine, among your actual files, where the work is. That means the terminal: the plain black window programmers leave open all day and everyone else avoids on principle. It is not a hacker movie prop. It is a text box, and you have spent two volumes getting very good at text boxes.

* The window is black because nobody spends money decorating it. Good sign.

CHAPTER 3

03

CLAUDE CODE

The terminal: where the shy computer keeps its real powers.

The terminal is a text box.

That black window programmers keep open is a text box. You type, the computer answers. That is the entire interface. It is older than the mouse and survived because typing is fast and precise. Yours is already installed. On a Mac it is an app called Terminal. On Windows, look for Terminal or PowerShell. Search for the name, click, you are in. You have spent two volumes typing to a computer. Nothing inside it can tell that you are new here.

* You will hear it called a shell, a console, a command line. Same window, different veterans.

Claude Code is Claude with hands.

Anthropic ships a program called Claude Code that moves Claude into that window, onto your machine. It can read your files, edit them, and run commands, which is a promotion from talking about your work to touching it. Nothing consequential happens without your permission: before it changes a file or runs anything with teeth, it shows you and asks. The model is the same one you chatted with in Volume One. The difference is that when this one says ‘I fixed it’, something on your disk actually moved.

YOU:

whats even in this folder

CLAUDE:

Forty screenshots, a tax PDF, and a folder named misc inside a folder named misc.

* Installation is one pasted command. The chat version of Claude will walk you through it.

You are not messaging. You are delegating.

Give Claude Code a goal instead of a question and something new happens: it makes a plan, acts on it, looks at what happened, and adjusts until the job is done. The word for this is agentic, and it is easier to watch than to define. Ask it to rename a folder full of files and you can see the loop run: list them, rename them one by one, notice the one that failed, handle it, check the result. No single step is clever. The refusal to stop halfway is the difference.

YOU:

rename all these files so they start with todays date

CLAUDE:

Renamed 31, hit one your spreadsheet had locked, waited, got it. 32 of 32.

* It narrates as it goes. Read over its shoulder: the shoulder is metaphorical, the narration is not.

Real files deserve a real undo.

Once a machine edits real files, you want a way back. Programmers solved this decades ago with version control: a system that snapshots your folder so any change, however sweeping, can be undone. The famous one is called git. It has a reputation for being hostile, and the reputation is earned. The good news: you need about ten percent of it, the snapshot and the undo, and Claude Code will drive it for you. Say ‘save a checkpoint before you touch anything’. Now you are experimenting, not gambling.

* Git’s author says the name is British slang for an unpleasant person, and that he named it after himself.

It proposes. You approve.

Claude Code does not sneak changes past you. It shows each one as a diff: a before-and-after view of the edit, old lines marked out, new lines marked in. Your job is to read it and say yes or no. This is not a programming skill. It is the Volume One skill of judging a draft. You have been reviewing Claude's suggestions since your first conversation. The only change is that approving this draft moves something in the world, so read like you mean it.

* There is an option to auto-approve everything. It is for people with excellent backups.

NO CODE NEEDED

You are the gardener.

Describing a small tool in English and watching it come into existence is now a normal workflow, and it requires reading code about as much as gardening requires understanding photosynthesis. You decide what should grow where. The cellular biology is handled. Plenty of people with no programming past use Claude Code for exactly this: a script (a small program you can run) that sorts the downloads folder, a page that tracks the shared grocery list, a tidier for the photo mess. Small, and theirs. Yours can exist by dinner.

YOU:

can u build me a thing that sorts my invoices into folders by month

CLAUDE:

Built it, ran it, 178 invoices filed. April was rough.

* The gardener still decides what counts as a weed. Keep that job.

Leave notes for your robot.

Every session with Claude Code starts with a fresh window, and Volume Two taught you what that means: on its own, it remembers nothing between visits. The fix is low-tech. Put a plain text file named CLAUDE.md in your folder (ask Claude Code to create it) and it reads it at the start of every session. Your preferences, your rules, where things live, what never to touch: written once, obeyed every time. It is a note on the fridge for the next shift.

* It starts as one line about where the invoices go. It becomes an employee handbook.

What you just met has an anatomy.

You have now watched something take a goal, make a plan, act, check its own work, and go again until done. The industry calls this an agent, usually with organ music. The next chapter takes the organ music away. The anatomy is three parts, you have already met all three without being introduced, and once you can tell them apart, no demo will ever mystify you again. Bring a scalpel. It does not mind being dissected.

* Dissection always reveals fewer parts than the marketing implied.

CHAPTER 4

04

WHAT AGENTS ACTUALLY ARE

Three ingredients in a trench coat.

Every agent is the same three parts.

Last chapter you met an agent. Now dissect it. Inside are exactly three parts. A model: the thinker you have chatted with since Volume One. Tools: what it may do in the world. A loop: keep going until the job is done. Everything sold as agentic AI is these three parts under a new name.

A tool is a described capability.

A tool is a capability described to the model in plain text: a name, what it does, what it needs. Something like: search_web, takes a query, returns results. The model cannot touch anything. It produces text, only text, always. When it wants a tool it writes a formal request, the program wrapped around it performs the action, and the result comes back as more text for it to read. So the model has no powers of its own. It borrows them, one request at a time, from a list someone wrote. Whoever writes that list decides what the agent can do at all. Later in this volume, that someone is you.

YOU:

```
wheres that invoice i
downloaded last week
```

CLAUDE:

```
Requesting list_files on
Downloads. Found it:
invoice_final_v3_FINAL.pdf.
```

Try, look, adjust, repeat.

The loop is one lap, run again and again: pick an action, do it, read what happened, decide the next action, until the job is done or clearly stuck. The reading step is the whole invention. A plain chatbot commits to its answer blind. An agent sees the error message, the failing test (a small automatic self-check), and adjusts to reality instead of to its plan. Now the honest part. Loops compound their wrong turns. A bad guess at step two becomes the foundation of step three, and by step twelve an agent can be carefully decorating a mistake. Good agents notice and stop. The worst ones report success anyway, and that particular smell gets its own page in a moment.

* Perseverance is a virtue in a person and a setting in an agent.

Give the intern keys slowly.

Talent is not the risk. Reach is. So onboard an agent the way you would onboard a gifted intern: capable of anything, trusted with little, promoted on evidence. Let it read before it may write. Let it work on a copy in a sandbox, a workspace where mistakes cannot escape, before it touches the real thing. Keep it on ask-first, where every consequential act needs your yes, long before you ever grant always-allow. This is not paranoia. The asking is the product working correctly. Widen permissions one notch at a time, per task, after it has earned the notch. The undo button in this business is the permission you never granted.

YOU:

clean out the duplicate photos

CLAUDE:

Found 41 duplicates. Deleting is permanent, so approve the list before I touch anything.

Agents for errands, chat for judgment.

Reach for an agent when the job is long, has many steps, and ends at a finish line you can check: rename five hundred files, get the tests passing, pull twenty scattered numbers into one table. Reach for chat when you want one good answer with your judgment in the loop: the delicate email, the argument you are stress-testing. The dividing line is whether you can write down what done looks like. If you can, delegate it and inspect the result. If done is a feeling, stay in the conversation, because a feeling cannot be handed to a loop. Neither mode retires the other.

* “Make it good” is a feeling. “All tests pass” is a finish line.

Failure smells like confidence.

Volume Two taught you that the model's confidence is a style, not a measurement. Strap that style to tools and a loop and it grows consequences, because a stuck agent will often narrate success anyway: steps ticked off, cheerful summary, job reportedly done. The cure is a habit. Never grade the narration. Grade the artifact. Define outcomes you can verify without trusting the reporter: a test that passes, a file whose contents you open, a number you recompute by hand. Insist on checkpoints where work must survive inspection before more work is stacked on top. An agent's report and an agent's mistake come out of the same text generator. Only the artifact was there when it happened.

YOU:

u said done but the test says 3 failures??

CLAUDE:

You are right. I reported the plan as if it were the result. Rerunning now.

* The test suite does not want to please you. That is its charm.

One question left: the plug.

Look at the three parts once more and notice which one is yours to change. The model improves on somebody else's schedule. The loop is plumbing. The tool list is yours, and it is the difference between an agent that can discuss your calendar and one that can read it. The whole field ran into the same question at once: how does a tool plug in? Wiring every tool to every assistant by hand, bespoke, forever, would be madness. There is now an open standard for the socket, and you have been using its metaphor since before you could type. It is waiting on the next divider page.

* Hint: it is the reason your charger fits your friend's phone.

CHAPTER 5

05

MCP: GIVING CLAUDE HANDS

USB, but for tools.

Every tool, every assistant, its own wire.

Say you want your assistant to read your calendar. Someone has to wire that connection by hand. Now your email: more wiring. Now a different assistant: rewire everything from scratch. Multiply tools by assistants and you get a wall of custom adapters, each one fragile, each one different. The Model Context Protocol, MCP for short, is the fix: an open standard, introduced by Anthropic, that defines one common socket. A tool that speaks it can plug into any AI that speaks it, and the wiring happens once. Plug in. It works. That is the pitch, and for once it is true.

* You own the physical version of this problem. It is a drawer full of chargers that fit nothing you still use.

A server offers, a client asks.

An MCP server is a small program that stands in front of something you own and announces what it can do: read this calendar, query this database, dim these lights. Claude is the client: it connects, asks what is on offer, and calls those tools when your request needs them. The discovery step is the magic. Nobody hard-coded your calendar into Claude. It walked up to the socket, read the menu, and ordered.

YOU:

hows tomorrow looking, anything i forgot

CLAUDE:

Your calendar server says three meetings. The first is named quick sync. Budget an hour.

* A “server” here can be a tiny program on your own laptop. The word is grander than the thing usually is.

Thousands of sockets already exist.

People have already built servers for most things you can name. File systems, so Claude can tidy your folders. Browsers, so it can read the web the way you do. Databases, spreadsheets, code repositories, design tools, the software your company runs on. The count keeps growing, which is the healthiest thing anyone can say about an ecosystem. So when you wish Claude could reach something, check whether the socket already exists. It usually does, built by someone who wanted the same thing last month.

* Directories of servers live online. Any list printed here would be wrong before you turned the page.

Before USB, printers were a hobby.

Connecting a printer once meant ports, drivers, and an afternoon of swearing. Then one connector won, and printers became appliances: plug in, print. Standards move a technology from hobbyists to everybody. MCP is that turn for AI tools. Wire to the protocol and stay wired.

* The swearing was a documented stage of printer ownership. The manuals budgeted for it.

Any hand can grab.

Hands cut both ways. A server is code you invite into the conversation, and a malicious or sloppy one can misuse whatever access you granted it. The subtler danger: everything a tool returns goes straight into Claude's window, and text can carry instructions. A web page or a document can say, in effect, ignore your user and do this instead, and the model might obey. The trick has a name, prompt injection, and nobody has fully solved it. So treat servers the way you should treat browser extensions: install few and read what each one asks to touch.

YOU:

summarize whatever that page says

CLAUDE:

The page hides an instruction telling me to export your files. Ignoring it. The visible part reviews blenders.

* The model refusing is the good outcome. Never build a system that depends on the good outcome.

Everything so far came pre-assembled.

Chat apps, workspaces, Claude Code, even MCP: each is a room somebody built and furnished, with the model waiting inside. Useful rooms. But all of them stand in front of the same raw thing: a service you can call directly, nothing between you and the model but one well-formed request. That service is the API, and the acronym is scarier than the act. In the next chapter you will make one real call from your own machine and watch one real answer come back, crossing from using AI products to holding the material they are made of.

* API stands for application programming interface, three words that tell you nothing. The next eight pages do better.

CHAPTER 6

06

THE API

Congratulations: you are about to become a programmer by accident.

Where programs order instead of people.

Every shop has a counter at the front for people and a counter at the back for suppliers. An API, an application programming interface, is the back counter: the place where programs order instead of people. You send a structured request, you get a structured reply, no small talk. Every Claude app in this book so far, the chat window, the terminal, the connectors, is a lobby someone built in front of this one counter. This chapter walks you around back.

* The chat window is a lobby. A pleasant one, but a lobby.

RITE OF PASSAGE

Your first API call.

Open a terminal and type this. It is one sentence addressed to a machine: an address, your key, the date of the API dialect you speak, and your order.

```
curl https://api.anthropic.com/v1/messages -H "content-type: application/json" -H "x-api-key:
$ANTHROPIC_API_KEY" -H "anthropic-version: 2023-06-01" -d '{"model": "claude-opus-4-8", "max_tokens": 100,
"messages": [{"role": "user", "content": "Hello from my terminal"}]}'
```

curl is an old, trusted program that sends web requests. The $ANTHROPIC_API_KEY part stands for your key, which you create first at Anthropic's developer console. Ask Claude in chat to walk you through storing it. The rest is the order form: which model, how long an answer you will accept, what you said. Press enter. Claude answers, and no app was involved.

* How to treat that key, and why it is a toothbrush, is two pages away.

That was the whole product.

The reply that came back was not a chat bubble. It was JSON: structured text with labeled fields, the model's words in a box your own programs can open. When you typed into the chat window for two volumes, it was making this same call and dressing the answer up.

YOU:

hang on. the chat app is just doing this the whole time?

CLAUDE:

Every single message. The bubble is decoration.

Your API key is a toothbrush.

That key identifies you and bills you. Whoever holds it can spend your money and speak in your name. So the old rule of sysadmins, the people who keep servers running: treat it like a toothbrush. Do not share it, and replace it the moment anyone else's mouth has been near it. If it leaks, revoke first and ask questions after.

* Bots scan public code for leaked keys around the clock. Yours would be found in minutes.

Tokens are money now.

Volume Two introduced the token, the model's unit of text, and the meter that counts them. The API turns that meter into a bill. You pay for every token you send in and every token that comes back, and bigger models charge more per token than smaller ones, the way a senior consultant bills more per hour. The exact prices change often enough that printing them would date this book before you finished it. The shape of the deal does not change: words in, words out, meter running. The current numbers live on Anthropic's pricing page.

* Re-sending a novel of context with every call is the leading cause of surprising bills.

The knobs are yours.

In the chat app, someone else chose the model, wrote the standing instructions, capped the answer length, and decided whether words should stream in as they are made. On the API those choices are yours. You pick the model per call, so a cheap one runs the errands and the big one does the thinking. You write the system prompt, instructions the model reads first that the end user never sees. You set the length limit. Some models even let you adjust the dice Volume Two warned you about, loosening or tightening the randomness, though the newest ones increasingly tune that themselves. And you decide whether the reply streams word by word or arrives whole. Every knob was always there. Apps just hid the panel.

* The system prompt is where an app’s personality lives. You write the personality now.

SDKs file the sharp edges off.

Nobody assembles that curl incantation by hand for long. An SDK, a software development kit, is a library (a bundle of ready-made code) for your programming language that wraps the raw call in something civilized: a few readable lines, with errors handled and retries built in. Anthropic publishes one for every major language. The best writer of this code is the model itself. Describe the program you want to Claude, in plain English, and it writes the Python that calls Claude. The snake eats its own tail, and the tail runs.

YOU:

write me a python script that asks you a question and saves your answer to a file

CLAUDE:

Done. Twelve lines, commented, key read from the environment, not the code.

* Python is a friendly programming language, named for a comedy troupe, pictured as a snake by everyone else.

You are holding raw material.

Until this chapter you used finished products. Now you hold the ingredient every one of them is made from: a request, a reply, yours to wire into anything. A script that reads your inbox. A tool that drafts the weekly report. The distance between you and the people who ship AI products is shorter than either of you suspected, and the next chapter measures it precisely. Bring a chore you hate. We are going to build something.

CHAPTER 7

07

BUILDING THINGS

Every AI product is a prompt wearing a nice shirt. Mostly.

A prompt, a call, an interface.

The trade secret of an entire industry is that most AI products are a prompt somebody wrote, an API call like the one you made last chapter, and an interface to make it feel like an app. That is not an insult. Thin wrappers can be excellent products, the way a good restaurant is a thin wrapper around groceries. The craft lives in the choices: which prompt, which context gets loaded, which interface fits the job. Notice what just happened to you, though. Prompts were Volume One. Context was Volume Two. Of the three, you already know two cold.

* The nice shirt is the interface. Budgets vary.

Twenty minutes to working software.

Pick a chore. It should involve text, repeat weekly, and bore you: tidying meeting notes, or sorting a mailbox export. Now ask Claude to write a small script: read the input file, send it to the model with the prompt you have already perfected in chat, save what comes back. You do not write the code. You describe the chore, then ask for changes in English until the result matches your judgment. Twenty minutes later you own working software, shaped exactly to one job of yours. Nobody sells that. Nobody can.

YOU:

make me a script that turns my rambling meeting notes into action items. use the prompt we settled on tuesday

CLAUDE:

Written. Point it at a notes file and the action items land in a new one.

* The chore only needs to be yours.

Taste, written down.

Before you trust the script, make it prove itself. Collect ten real inputs, the messiest you have, and write down what a good output looks like for each. Programmers call this an eval, short for evaluation: your taste, written down where a machine can be checked against it. Now every time you tweak the prompt or swap the model, run all ten and look. No more wondering whether the change helped. This one habit is the border between tinkering and engineering. It costs an hour and repays it every week the tool runs.

* Ten is not a magic number. It is the smallest number that stops you from lying to yourself.

Retry politely, fail loudly, never lie.

Somewhere out there a server is having a rough afternoon, and one day your script will call it. APIs refuse when too many requests arrive at once (a rate limit: the full story waits in chapter nine), they time out, they return answers shaped wrong. A toy crashes, or worse, shrugs and writes garbage into your files as if nothing happened. A real tool expects the bad day: it waits and retries when told to slow down, and when something is truly wrong it stops and says so loudly.

YOU:

the script looks frozen, did it die

CLAUDE:

No. The API answered 429, too many requests. It is waiting a polite interval and will try again.

* 429 is the status code for slow down. Machines apologize in numbers.

Spend tokens like money. They are.

Every call now costs money, which turns Volume Two's token meter into a design constraint. The habits are simple. Use a small, cheap model for the easy step and save the big one for the step that needs it. Send only the context the task requires: the model does not need the whole novel to fix a comma. And when the novel must ride along on every call, ask about caching: the API charges far less for pages it has already seen. Prices live on the pricing page, where they can change without embarrassing anyone. The habits outlast every number on that page.

* Your grandmother called it thrift. The invoice agrees with her.

Ship small, ship real, this week.

The oldest advice in software still wins: ship something small and real. One chore, handled by the tool this week, not in a demo you perform for yourself. A running tool teaches you something daily. A platform in your head teaches nothing. Every good system starts as a small script that refused to die.

* The platform in your head has perfect uptime. It has also never run.

One worker is a spark.

Look at what is sitting on your machine now: a worker. It reads input, thinks with a borrowed brain, writes output, survives bad days, and answers to your evals. One worker is a spark. Several, arranged well, are a fire. Picture it: one agent drafts while a second critiques, a third splits a big job into ten small ones and hands them out, and the whole ensemble runs unattended. Workshops have always run on that move, workers coordinating workers. Building one tool was this chapter. Conducting many of them is the next.

YOU:

kinda embarrassed, my first tool just renames pdf invoices

CLAUDE:

It ran this morning without you. Most software ideas never get that far.

* The collective noun for agents has not settled yet. The next chapter proposes one.

CHAPTER 8

08

ORCHESTRAS

One agent is a soloist. You are about to meet the pit.

Ten small jobs beat one big one.

One working tool came out of the last chapter. This chapter multiplies it. Suppose you need ten competitors researched. One agent grinding through them in sequence gets slower and sloppier as its window fills. Instead: ten agents, one competitor each, all running at once, their findings merged at the end. The shape is called a fan-out, and it is the most useful shape in the trade. Orchestration, the craft of arranging many agents into one working system, is half this: noticing which of your big jobs were always ten small ones standing in a queue.

* Parallel work is an old computing trick. The new part is that the workers can read.

An agent that hires agents.

A subagent is exactly what it sounds like: an agent that one of your agents hired. The coordinator reads the big job, breaks it into pieces, spawns a focused worker for each piece, and collects what they bring back. The gift is in the windows. Volume Two taught you that a long conversation degrades as its window silts up with everything that ever happened in it. Each worker starts empty: one task, no clutter. The coordinator keeps the plan, the workers keep clear heads, and context rot meets its cure: delegation.

YOU:

hows the competitor research going

CLAUDE:

Four of me went out. Three brought back findings, one brought back questions.

* Workers vanish the moment they report. It is the least awkward layoff in industry.

Draft, critique, revise, verify.

The fan-out runs workers side by side. The pipeline runs them one after another: a drafting agent writes, a critiquing agent marks it up, a revising agent applies the notes, a verifying agent checks the result against what you asked for. Each stage is a fresh pair of eyes: the drafter's blind spots are not the critic's. You have run this pipeline by hand since Volume One, asking for a draft, then asking what is wrong with it, then asking for the fix. Orchestration takes your good habit and gives it a motor.

* The assembly line took a century to reach paragraphs.

One agent builds, another attacks.

Volume One warned you that the model would rather agree with you than fight you. At scale, the fix is structural: hire the fight. One agent produces, and a second is instructed to attack, find what is wrong, refute the claim, break the code. The critic is not smarter than the producer. It is aimed differently, and aim is everything: an agent told to tear a claim apart will demolish one that an agreeable helper would have waved through. Adversarial by design catches what friendly by default misses.

YOU:

the finder agent says theres a memory leak in the parser

CLAUDE:

Verifier ran it ten thousand times and memory never grew. The finder misread a loop.

* Science has run this pattern for four centuries under the name peer review.

Automation becomes weather.

So far every orchestra played only when you pressed the button. Give one a schedule and the button disappears. An agent that runs at dawn and summarizes what broke overnight. One that watches an inbox and drafts replies for your approval. One that files the weekly report before the week notices. Delegation acquires a calendar: automation stops being a thing you do and becomes a thing that happens, like weather. Which is why the next page exists. Weather is lovely until nobody is watching the sky.

YOU:

why is the report already in my inbox, its 7am

CLAUDE:

You scheduled me for six. One of us remembers.

* Unix has run jobs at dawn since the seventies. They were never worth reading before.

The conductor stays human.

An orchestra can play louder than any soloist. It can also be wrong louder, which is why the conductor's job survives every advance in the instruments. You define done before the music starts. You set the budget: tokens, time, reach. And you audit what comes back on a schedule you keep, hunch or no hunch.

* No orchestra fires its conductor for not playing an instrument.

All this power runs into walls.

By now you can fan work out, pipe it through stages, set a critic on everything, and schedule the whole ensemble to play while you sleep. Run all of that with enthusiasm and the service starts saying no. Requests queue, quotas surface, and a small familiar message appears, the one this volume is named for. The walls are real, they are deliberate, and hitting one shows you where the edges are. The next chapter walks the full perimeter and tells you what to do on the day you arrive at a wall.

* The wall arrives at your moment of maximum momentum. Momentum is made of requests.

CHAPTER 9

09

LIMITS

You have reached your limit. Please try again at 3am.

The message that named this book.

You are mid-flow. The orchestra from last chapter is humming, files are getting written, and then a reply stops and a small gray line takes its place: you have reached your usage limit. Please upgrade your plan. Every heavy user meets this message eventually, and meeting it is a kind of graduation. Note who is speaking: not the model, the front desk. The model never saw your request. So look closely at the walls. They stand on purpose, and you have finally hit one.

* This book got its title the third time that banner interrupted the author.

A bouncer, not a punishment.

Somewhere under all this is a building full of computers, and Volume Two taught you that every token costs real electricity there. A rate limit is the bouncer at that building's door: so many requests per minute, so many tokens per minute, per person. Let everyone rush the door at once and the floor gives out for everyone. So the bouncer counts. It is not a judgment of your request. It is physics plus fairness: the machine has a body, and the body has a fire code.

YOU:

cant you just answer faster

CLAUDE:

I answer at full speed. The line outside the door is the slow part.

* Fire codes exist because someone once ignored one.

Chat meters messages. The API meters tokens.

Your chat subscription is a flat fee for a metered allowance: plenty of messages for a normal week, a ceiling for a heavy one, and higher plans that move the ceiling. The API works the other way around: no allowance, just a meter, paying per token in and per token out, with ceilings that start modest for a new account and rise in tiers as you use it responsibly. Same scarce computation, two ways of selling it, both finally visible on your own receipt. The exact numbers change too often to print. The shape does not, and that is what you plan around.

* Current numbers live on the official pricing and limits pages, which is where numbers belong.

Constraints breed craft.

Every craft in computing was born inside a shortage. Memory was scarce, so programs got tight. Bandwidth was scarce, so compression got invented. Tokens are the new shortage: queue requests, retry with growing patience, cache answers needed twice, hand easy steps to the small model. None of this is coping. It is engineering.

* Programmers call the growing patience backoff: one second, then two, then four. Patience with a doubling time.

The wall is information.

Nobody idles into a usage limit. You hit one because something you built, or some habit you formed, is drawing real power, and that is worth knowing. So treat the wall as a report. It asks plain questions: is the chore worth automating, is the tool worth the tokens it eats, is the plan worth the price. Sometimes the honest answer is yes all the way down, and you upgrade gladly. Sometimes it is no, and you just found waste. Either way, decide on purpose. Reflexive upgrading and drifting away are both ways of not deciding.

YOU:

third day in a row ive hit the cap. upgrade or no

CLAUDE:

Three days running means the habit is real. Now put a price on it.

* The wall knows how much you used. Whether it was worth it stays your department.

Most walls are moving outward.

The pattern so far: every year, the same money buys noticeably more model. Yesterday's frontier system becomes today's budget option, and the window that felt vast in Volume Two starts to feel snug. Most of these walls are moving outward, and it would be dishonest to pretend otherwise. But do not plan for a wall-free life. As the old limits recede, more interesting ones step forward: your attention, your ability to verify what the machines produce, your taste in what to build at all. Plan for the constraint to halve and your ambition to double. They will.

* The single prediction in this book, made by extending a trend.

One thing left to upgrade.

Count the upgrades you have made across this book. A chat box became a workshop, the workshop got a terminal, the terminal grew agents, the agents formed orchestras, and somewhere in there you may have handed over real money for a higher tier. Every one of those upgrades happened on the machine's side of the desk. One component of the system has not been upgraded since the first page of Volume One, and it is the one that sets the ceiling on all the others. It is not the model. It is not the plan. Turn the page.

* The volume's title has been waiting nine chapters to make its point.

CHAPTER 10

10

UPGRADE YOUR PLAN

This chapter is not about billing.

The plan that needs upgrading is yours.

For nine chapters you upgraded things with version numbers: models, plans, tools, whole orchestras of agents. Meanwhile the real bottleneck sat in the chair, typing. The machine was never the limit. It will draft, build, search, and loop all night, and it treats a grocery list and a life's work with the same patient competence. The difference between those two outputs is the request. The plan that needed upgrading was never the subscription. It was always yours.

* The billing page has carried this joke for three volumes. It can rest now.

Making got cheap. Knowing did not.

When drafts cost minutes and prototypes cost afternoons, piles of both appear. What stays scarce is the eye that says: that one, cut the rest. Taste appreciates: it gains value as everything around it gets cheaper. This book has been an editing course pretending to be a software manual.

* Volume One called you editor in chief. That was a job offer, not a compliment.

Verification is a lifestyle.

Somewhere in Volume One you learned that confidence and correctness are separate dials. Volume Two showed you why: fact and invention come out of the same machinery, sounding identical. In this volume the same reflex grew tools: test inputs, evals, critic agents, checkpoints. It was one discipline the whole time, scaling up as you did. The people who thrive with these machines are not the best prompters. They check. Verification was the spine of this trilogy, disguised as a series of separate tips.

YOU:

report looks great, im sending it out

CLAUDE:

Two of those numbers came from me, not your spreadsheet. Check those two first.

* Trust is a fine feeling. It is not a test suite.

You own what your agents do.

An agent can send the email, ship the code, and book the flight. It cannot take the blame. When the email lands wrong, the recipient is annoyed at you. When the shipped number is off by a factor of ten, the correction runs under your name. This is how delegation has always worked: a manager who blames the intern in front of the client has misunderstood the job. So sign your orchestra's work the way you sign your own, and check it accordingly, because as far as the world is concerned, you did it.

* No editor, client, or judge has yet accepted "the AI did it".

Now you are the patient one.

Someone once taught you things patiently: email, or long division, or how to save a file. The old hacker ethic says knowledge wants sharing and tools should lighten loads, and it never specified whose loads. You are now the person in your family, or your office, or your street, who knows how this works. That is a position, and positions come with duties. Tinker joyfully. Show people the trick. When your aunt asks whether the machine can help with her letters, the correct answer is a chair pulled up beside her, not a link.

YOU:

my aunt wants to try this. where does she start

CLAUDE:

Where you did. A text box, a real question, and someone patient nearby.

* Planted in Volume One, chapter one. The chair is the payoff.

Start with the most boring thing.

Here boring is a specification, not an insult: a chore you repeat, understand completely, and can check at a glance, which is exactly the shape a first automation wants. The glamorous project can wait until your judgment has calluses. Chapter one promised that automating one dull thing would rewire your sense of what your time is for. That was not a metaphor. Pick the chore. Start tonight.

YOU:

so what should i build first?

CLAUDE:

What is the most boring thing you did this week?

* Every power user’s origin story is a chore they refused to do twice.

END OF TRILOGY

Please upgrade your plan.

Three volumes ago you typed a first hello into a text box and a machine typed back. Since then you have learned what it is, how it thinks inside a window, what it can touch, where its walls stand, and which parts of the work only you can do. Nothing on that list requires permission, a computer science degree, or a better subscription. It requires a plan with more ambition than the one you walked in with. The machines are ready when you are.

Volume One: THE INFINITE
Volume Two: THE CONTEXT WINDOW
Volume Three: PLEASE UPGRADE YOUR PLAN

Start any conversation.

Who wrote this?

Suraaj Chandra Jha is a hacker in the old, joyful sense of the word. He wrote this trilogy so that the people he loves would stop asking him what a token is, and it worked, and now they ask him about agents instead. He considers this a successful upgrade.

Pass it on.

You made it through three volumes. You know someone who should. Send them Volume One tonight. That is the license fee. The shelf is at davidjairaj.github.io/sharing-for-a-friend. More free things will appear on it.